

ProjectFlow Assessment — Interview Prep

A private study guide for the technical interview on this submission. Not part of the deliverable — this is for you. Organized as: a short opening pitch, then Q&A by topic, then a "gotcha questions" section for the things a sharp interviewer is likely to push on precisely because they're judgment calls, not facts.

Ground rule for the interview: **you can always point back to a specific file**. Every answer below names the file/function it's grounded in. If you get a question this doc doesn't cover, the honest move is "let me think through that against the actual code" — not guessing.

0. The 60-second opening pitch

If asked "walk me through what you built":

"The brief had four real pieces of work: build task assignment end to end with three business rules, add an activity feed for assignee changes, investigate and fix a reported authorization bug, and fix a concurrency bug in task numbering. I started by reading the existing codebase fully before writing anything — the auth model, the existing `ProjectAccessService`, how comments were already built, since that's the closest existing pattern to what I needed for activity. Found the authorization bug had a very specific root cause: one endpoint, `updateStatus`, was the only task-mutating method that never called the project-access check every other method already calls. Fixed the concurrency bug with an atomic per-project counter instead of the read-then-write count that was racing. Built assignment with the three rules enforced server-side, an activity collection separate from Task for the reasons I can go into, and a frontend selector with optimistic updates. Wrote four test files, documented the reasoning in `ASSESSMENT_NOTES.md`, and was honest in `AI_LOG.md` about the one thing I couldn't verify — the sandbox I built this in couldn't download the MongoDB test binary, so the tests are hand-verified against the service logic but not machine-executed by me."

That last sentence is important — **say it before they ask**. Volunteering the one gap you know about reads as rigor, not weakness. Burying it and having them find it reads as evasion.

1. System understanding

Q: How is the application structured? Monorepo (pnpm + Turborepo): NestJS API, Next.js App Router frontend, a shared package of types/enums both import. API is one Nest module per domain concept (`auth`, `projects`, `tasks`, `task-activity`, ...), each with controller → service → Mongoose schema, DTOs validating at the boundary. Frontend mirrors it with `features/<domain>/{api, hooks, components}`.

Q: Where does business logic live? Services, never controllers, never schemas. Controllers only extract request data and call a service method. This is why `TaskService.assign` reads as one linear story — every rule, in one place, not scattered across a controller and a schema hook.

Q: How does the frontend talk to the backend? How is server state handled? One `apiRequest` wrapper (`lib/api-client.ts`) owns the base URL, the auth header, error parsing. Every domain wraps it in typed `api.ts` functions, which TanStack Query wraps in `hooks.ts`. Server state lives only in the Query cache — no component holds a local copy in `useState`. Query keys are centralized (`lib/query-keys.ts`) so invalidation can't silently drift from what a query elsewhere reads. I extended this pattern rather than inventing a new one: `queryKeys.taskActivity(taskId)` sits right next to `queryKeys.task(taskId)`.

Q: How are auth and authorization implemented? Two separate concerns, deliberately. **Authentication:** JWT bearer token, `JwtAuthGuard` registered globally via `APP_GUARD`, routes opt out with `@Public()`. **Authorization:** centralized in `ProjectAccessService`, which answers "may this user touch this project, and as what role?" from two independent sources — an elevated org role (`OWNER`/`ADMIN`, grants access org-wide) or an explicit project-membership row (`PROJECT_MANAGER`/`MEMBER`, scoped to one project). `assertCanView` gates reads/routine mutations, `assertCanManage` gates structural changes.

Q: How are the entities related? `User` → `OrganizationMember` (role) → `Organization` → `Project` → `ProjectMember` (role) → `Task` → `Comment` / `TaskActivity`. Membership is its own collection on both levels, not an embedded array, specifically so it's indexable/queryable directly (e.g. "every project this user is in") without unwinding arrays inside parent documents.

Q: What risks did you find that you didn't fix? Three worth having ready (full list is in `ASSESSMENT_NOTES.md` §1):

1. Authorization is enforced *by convention* (every method remembers to call the access check), not structurally. That's the actual root cause behind both the reported bug and the code-review exercise's missing checks — same shape of mistake, twice. I'd fix this with a route-level guard given more time (see "two more days" list) — didn't do it now because it's a refactor touching every task/comment endpoint, more change than either individual fix warrants.
 2. `ProjectAccessService` costs 1-2 Mongo round trips per call, and every task/comment endpoint calls it at least once. Fine today; worth watching once activity reads get frequent at scale (tie this into the scaling answer if asked).
 3. Task deletion cascading to comments/activity is also convention-based (the service remembers to delete both) rather than enforced at the DB layer.
-

2. Task assignment & business rules

Q: Why `assigneeId` directly on `Task`, not a separate assignment entity? A task has *at most one* assignee at a time — nothing in the brief asks for concurrent multi-assignee or "assignment as its own historical record" (that's what the activity feed is for). The brief's own concept notes describe it as a single nullable reference, explicitly distinct from `createdBy`. A join table would model a relationship that doesn't exist here.

Q: Walk me through the three rules and how they're enforced.

- **Rule 1** (assignee must be a project member): checked via `ProjectMembersService.findRole` — if it returns nothing, `400`. I chose to require an *explicit* membership row, not "can this org owner reach this project" — deliberately, see the ambiguity answer below.
- **Rule 2** (who may assign whom): `canManage(access)` — reused directly from `ProjectAccessService`, not a new predicate, because the set of roles it already recognizes (`OWNER`/`ADMIN`/`PROJECT_MANAGER`) is *exactly* Rule 2's privileged set. A regular member can only target themselves.
- **Rule 3** (unassignment): symmetric with Rule 2 — a privileged role can clear anyone's assignment; a regular member can only clear their own.

Q: Where was the brief actually ambiguous, and how did you resolve it? Rule 3. "An authorized user may remove the current assignee" doesn't say whether a regular member counts as "authorized" for *their own* assignment. I resolved it as symmetric with Rule 2 (self-assign implies self-unassign) rather than requiring manager privilege to unassign at all — the alternative would mean a member who self-assigned couldn't undo their own mistake without asking a manager, which reads as an unreasonable reading of "authorized." This is the one place I'd expect genuine pushback in the interview, and I have both directions tested (`task-assignment.e2e.spec.ts`).

Q: What if I told you Rule 1 should count elevated org roles as implicitly "project members" for assignment purposes? That's a defensible alternative reading — I chose the stricter one because the concept note says "project: scopes who may be assigned at all," which reads to me as membership being the actual gate, not administrative authority. If the real product requirement is "an org owner can assign themselves to anything," that's a one-line change (`findRole` OR `isPrivileged` in the Rule-1 check) — I'd want that confirmed as an actual product decision before making it, not assume it.

Q: What if product asked for multiple assignees tomorrow? `assigneeId: ObjectId | null` → `assigneeIds: ObjectId[]`. The activity metadata (`from`/`to`) would need to become arrays or per-member events instead of a single before/after pair. I'd raise that the "regular member can only assign themselves" rule gets more interesting with multiple assignees — can

they add themselves to an already-multi-assigned task without touching the others? — and get that clarified rather than assume.

Q: Why does a no-op assignment not create an activity row? Because it isn't a change. If assigning a task to its current assignee logged an activity entry, the feed would say "X assigned Y" every time someone re-saved the same value, which misrepresents the task's actual history to anyone reading it later. Tested explicitly (does not record an activity entry for a no-op assignment).

3. Activity history & API design

Q: Why a separate task_activities collection instead of an array on Task? Three reasons, in the order they actually mattered while deciding:

1. **Unbounded growth** — it's append-only, forever. An embedded array would make old, frequently-reassigned tasks' documents grow without bound, eventually risking MongoDB's 16MB document cap.
2. **Independent, paginated reads** — GET /tasks/:taskId/activity reads a page of the feed without loading the whole task document, and without \$slice gymnastics on an embedded array.
3. **Independent scoping for retention/archiving later** — a collection can have its own TTL index, its own archival job; a field embedded in another collection's documents can't.

Q: Why is projectId denormalized onto every activity row when it's already reachable via the task? So activity can be indexed and queried *per project* without a join back to tasks first. Doesn't matter today (only query is per-task); matters the moment activity is the largest collection and something project-scoped (archiving, a project activity feed) needs to run without joining through every task first. Small redundant-storage cost now, for a query path that becomes necessary at scale — flagged explicitly in the scaling section as the reason this was denormalized ahead of actually needing it.

Q: How did you avoid N+1 resolving actors? TaskActivityService.toEntries collects every distinct actor/from/to id across a whole page into one Set, then one UserService.findManyByIds call — never per-row. Same batching pattern already used in TasksService.toSummaries for createdBy, extended to also cover assigneeId rather than inventing a second approach.

Q: Why offset pagination and not cursor? Because the only query today is "one task's own activity," which is always small in practice (a task doesn't realistically accumulate thousands of assignee changes). Offset pagination's actual cost problem — walking and discarding skipped documents — only bites on deep pages into a huge result set, which doesn't describe this

query. I'd switch to cursor pagination the moment a cross-task activity view ships (my own "two more days" #2), since *that* query genuinely can be deep into a huge collection.

Q: What's the index, and why that shape? `{ taskId: 1, createdAt: -1 }` — matches the only access pattern that exists: "most recent activity for one task." If I added per-user or per-project activity views, those would need their own compound indexes added at that time, not speculatively now.

4. Frontend

Q: Why build a custom `DropdownMenu`-based selector instead of the existing `Select` component? Tried `Select` first, actually — it's the obvious existing form input. Rejected it once I took "searchable when appropriate" seriously: Radix `Select` only supports typeahead-jump, not real substring filtering, so a project with more than a handful of members would have an unusable picker. `DropdownMenu` was already a dependency (used for the user menu), so I built the filter input into its content instead of adding a new dependency.

Q: Explain the optimistic update flow. `useUpdateTaskAssignee`: `onMutate` cancels in-flight queries for that task, snapshots the current `TaskDetail` from cache, writes the new assignee into cache immediately. `onError` restores the snapshot — this is what happens visibly if, say, a regular member's UI let them try something the backend rejects with 403 (shouldn't happen given the UI already filters options, but the backend is the real enforcement, so this path exists for real). `onSettled` invalidates the task, its activity feed, and the project's task list — so the server-computed fields (`viewerCanAssignOthers`, the new activity row) become authoritative regardless of whether the optimistic guess was right.

Q: How does permission handling work end to end? Server computes `viewerCanAssignOthers` on `TaskDetail` (from the same `canManage` check used to enforce the rule). Frontend uses it only to decide what to *render* — a non-privileged viewer's selector only lists themselves as an option. The backend enforces the rule independently on every write regardless of what the frontend showed — this is the "frontend validation alone is not enough" line from the brief taken literally: the frontend flag is a UX nicety, never the actual gate.

Q: Why did you change the task-row avatar from creator to assignee? Judgment call, documented in `ASSESSMENT_NOTES.md`. On a board card, "who owns this right now" is more actionable at a glance than "who filed this originally" — closer to how Jira/Linear-style boards work. Noted as a trade-off rather than silently changed.

Q: Accessibility / responsive considerations? The selector trigger has `aria-label="Assignee"`, disables during the mutation (not just visually — the `disabled` attribute), and the activity list is a semantic `<ul>`/`<li>` under a labelled `<section aria-`

`label="Activity">`). Didn't do a full audit pass — if pushed, I'd name that as real scope I'd want more time for rather than claim more than I verified.

5. The production bug

Q: Walk me through root cause → fix. `TasksController.updateStatus` never extracted `@CurrentUser()`, and `TaskService.updateStatus` never called `ProjectAccessService` at all — zero authorization beyond "is this a valid JWT." Every sibling method (`create`, `update`, `remove`, `findOne`) calls `assertCanView`/`assertCanManage` before touching the task; this one didn't. Found it by reading the file top to bottom and noticing the asymmetry, not by guessing where bugs typically live. Fix: added the same `assertCanView` call and threaded `userId` through the controller.

Q: Why `assertCanView`, not `assertCanManage`, for the fix? Deliberate, not a shortcut. `assertCanManage` is reserved elsewhere for structural actions (delete a task, manage membership). Status changes are routine, collaborative Kanban actions — any project member moving a card is normal. Using `assertCanManage` would fix the bug but *also* silently lock regular members out of something they could do before, which isn't what was reported or asked for. `assertCanView` closes exactly the reported hole — "not a project member at all" — without changing behavior for anyone who already had legitimate access.

Q: How would you stop this class of bug from recurring? This is my #1 "two more days" item on purpose: a route-level authorization guard (`@RequireProjectAccess('view'|'manage')`) that resolves the project id from route params and runs the check *before* the handler executes at all — turning "did the author remember" into "the framework enforces it." Both this bug and the code-review exercise's missing checks are the same root cause, which is exactly why it's the highest-leverage fix on the list, not just a nice-to-have.

Q: Could this have been caught by existing tests? No — there was no test for `updateStatus` authorization before this assessment (comments and other endpoints had outsider-access tests; `updateStatus` didn't). That gap is itself informative: the existing test suite followed the same "test what you remembered to test" pattern as the bug itself. I added it now (`task-authorization.e2e.spec.ts`).

6. Concurrency fix

Q: Why is `countDocuments` + `+1` unsafe? Two separate operations, no lock between them. Two concurrent requests can both `countDocuments` and read the same number, then both write a task with `count + 1` — not a hypothetical, it's why the brief describes it as happening "occasionally."

Q: Why an atomic counter instead of retry-on-unique-violation? Considered it. Retry-on-conflict means every creation under real contention pays for a wasted round trip plus a retry loop that needs care to not retry forever. `findOneAndUpdate({ $inc })` gets the right number in **one** round trip, always, because MongoDB serializes concurrent writes to the same document at the storage-engine level — that's the actual atomicity guarantee I'm relying on, not application-level locking.

Q: Does this need a replica set / multi-document transaction? No — and this is worth being precise about if asked, because it's an easy place to sound less sure than you are: **single-document atomicity in MongoDB doesn't require a replica set or a session.**

`findOneAndUpdate` on one document is atomic on a lone `mongod` the same as on a replica set. Multi-document transactions are the thing that needs a replica set, and I deliberately didn't reach for one here — see the next answer.

Q: What about the task-creation write and the counter increment — those are two different documents. Is *that* atomic? Good catch if asked, and the honest answer is: not currently wrapped in a transaction. The number is reserved atomically (the counter increment itself can never collide), then the task document is created with that number. If the process crashed between those two steps, the number would just be "burned" — never issued to any task, a gap in the sequence, not a duplicate. I treat that as acceptable rather than a bug: the actual invariant that matters is *uniqueness*, not perfect density — the same way GitHub issue numbers have gaps and nobody considers that broken. I did *not* wrap this in a MongoDB multi-document transaction, because doing so trades a very rare, harmless-gap failure mode for a real operational requirement (transactions need a replica set) to prevent something that isn't actually a correctness problem. That's a judgment call I'd defend, not an oversight.

Q: Is there a similar "not actually atomic" gap in the assignment feature — the task update and the activity insert? Yes, and worth volunteering if the conversation gets here: `assign()` does `task.save()` then `taskActivityModel.create()` as two sequential operations, not a transaction. A crash between them would leave the assignee changed with no activity row recording it — unlike the counter/gap case above, this one is a real (if narrow) data-consistency gap, not a merely-cosmetic one, since the activity feed is meant to be a trustworthy record. I didn't wrap it in a transaction for the same reason — no existing code in this repo uses multi-document transactions, and introducing the first one is exactly the kind of architectural addition the brief says to justify carefully, for a failure window that's a process crash inside a few milliseconds of sequential Mongo calls. If pushed on it: it's a legitimate "would want before this ships to real production traffic" item, and I'd rather say that plainly than pretend it isn't there. (This isn't currently called out in `ASSESSMENT_NOTES.md`'s Known Limitations — flagged here because it's exactly the kind of thing an interviewer looking closely would find, and finding it yourself in the interview is much better than being caught by it.)

Q: What about the first-task-in-a-project race specifically? Two concurrent "first ever task" requests can both attempt to *upsert* the counter document — the one operation where MongoDB doesn't fully serialize for you, since there's no existing document to lock yet. One can lose with an `(E11000)` duplicate-key error on `(_id).nextTaskNumber` catches exactly that error and retries once, which becomes a plain (unconditionally safe) increment against the now-existing document. Tested directly (`(task-concurrency.e2e.spec.ts)` fires 20 concurrent creations against a brand-new project specifically to exercise this).

Q: Why also add a unique index if the counter already prevents duplicates? Defense in depth, not redundancy for its own sake — if I'm wrong about the counter logic, or a future change reintroduces the race, the unique index turns "silent data corruption" into "a loud `(500)` on the second write." Cheap insurance for a field that's supposed to be unique per project regardless of how the number was produced.

7. Database design

Q: What indexes did you add, and why each one?

- `(TaskCounter._id)` — implicit (it's the project id), no separate index needed; every lookup is a primary-key lookup.
- `(Task { projectId: 1, number: 1 })` — made **unique** (was previously non-unique); enforces the actual invariant at the DB layer.
- `(Task { projectId: 1, assigneeId: 1 })` — supports "tasks assigned to X in project Y," a natural query I anticipated the frontend or a future "my tasks" view would need.
- `(TaskActivity { taskId: 1, createdAt: -1 })` — the only query that exists today, newest-first.

Q: Embedding vs. referencing — how did you decide? Membership (org/project) is already referenced, not embedded, in the existing codebase — I followed that. Activity is referenced (separate collection) for the reasons above. The one place I embedded rather than referenced: `(TaskActivity.metadata) ({ from, to })` is a sub-document, not a separate collection, because it's small, fixed-shape, and only ever read/written alongside its parent activity row — there's no independent query pattern for "give me all `(from)` values across activity," so there's no reason to pay for a second collection and a join.

8. Testing

Q: Why these specific tests and not others? Started from the brief's own bullet list (it enumerates almost exactly the test cases I wrote) and added the ones a careful engineer would add next to it: the no-op-doesn't-log-activity case, the "org member with no project row" edge case on Rule 1, both directions of Rule 3's ambiguity, and a "sequence continues correctly after

a burst" test for concurrency (not just "no duplicates during the burst" — also "the counter is in the right state afterward").

Q: You said you couldn't run these. How do I know they're right? Fair question, answer it directly: I traced each assertion by hand against the actual service code — for every test, I can point to the exact line in `TaskService`/`TaskActivityService` that makes it pass, and I ran `pnpm typecheck`/`pnpm lint` against them (which catches a real class of mistakes — wrong types, wrong imports across the shared-package boundary — just not logic errors). That's real verification, but it's not the same as execution, and I said so in `AI_LOG.md` instead of implying otherwise. If asked to prove it live: `pnpm test` after cloning is the very first thing to run.

Q: What's not tested? Frontend components have no test coverage in this submission — the brief's own test bullet list is entirely backend/business-rules, and given the 6-hour cap, backend business-rule correctness plus the regression tests for the two real bugs took priority over frontend unit tests. I'd name that as a real, deliberate trade-off if asked, not something I overlooked.

9. Code review exercise

Q: Talk me through how you approached reviewing that snippet. Read it against the three rules from the brief first, since that's the actual functional spec the review is checking against — and the biggest finding fell straight out of that: `userId` is a parameter but never used for anything, so *none* of the three rules are enforced. That's not a style note, it's the same shape of bug as the reported production issue — worth saying explicitly in the interview, since drawing that connection shows you're pattern-matching across the whole assessment, not reviewing each part in isolation. Then worked outward: no activity record (breaks the feed's promise to be a complete record), no unassignment path at all (the signature can't even express it), unparameterized `NotFoundException` — which is a minor DX issue on its own but becomes a real information-leak when combined with zero auth (an attacker can tell "task doesn't exist" from "user doesn't exist" from "success").

Q: What would you *not* ask the engineer to change? The overall shape — fetch, validate, mutate, `.save()`, return — matches this codebase's existing style (Mongoose documents, not a repository abstraction). I wouldn't ask for a rewrite for its own sake; the brief explicitly says not to, and more importantly it's genuinely fine — the problem is missing behavior, not bad structure.

10. Scaling

Q: Give me the 30-second version. "The thing that actually breaks at 100x isn't write volume — writes to activity are already cheap, one small document per change. It's read *patterns* that don't exist yet: a per-user 'assigned to me everywhere' or per-project feed, which the single-

task index doesn't cover. I'd add those indexes and switch to cursor pagination *when* those features actually ship, not before. Beyond that: archive activity older than some retention window into cheaper storage once the collection is confirmed to be the largest in the system, move any *downstream* effects of an activity event (notifications, search indexing) onto a queue so they never add latency to the assignment request itself, and get query-level observability in place *before* any of this — so decisions like 'is the auth check hot enough to cache' are made from real metrics, not a guess."

Q: Why not just add Redis / a queue now, given you know it's coming? Because "I know it's coming" isn't the same as "it's a real problem yet," and the brief is explicit that adding infrastructure to look sophisticated is the wrong instinct. Caching (`ProjectAccessService`) checks, for example — I flagged it as a real future candidate, but I'd want to see it show up in actual slow-query/latency metrics before adding a cache invalidation problem to solve a cost that might be perfectly fine in practice.

11. AI usage & workflow

Q: How did you use AI tools here, and how do you know you actually understand what it produced? Used directly as a coding assistant with shell/file access to the cloned repo — not "generate a PR and skim it." Everything in (`AI_LOG.md`) is real: I can walk through *why* (`TaskActivityModule`) imports (`TasksModule`) rather than the reverse (avoiding a circular dependency, mirroring how (`CommentsModule`) already does it), *why* (`assign()`) normalizes (`task.assigneeId ?? null`) (a real bug I traced through — a raw driver-inserted test fixture doesn't get Mongoose's schema defaults applied, so the field can come back (`undefined`) instead of (`null`), which would've crashed (`.equals()`) on it), and *why* I rejected the first version of the assignee selector ((`Select`) doesn't support real filtering) and the first version of (`updateStatus`)'s fix ((`assertCanManage`) would've been over-restrictive). Those are the kind of answers you can only give if you actually traced the reasoning yourself, not memorized a summary.

Q: What's the biggest thing you'd want to defend if challenged? Probably Rule 3's interpretation (§2 above) — it's the one place the brief is genuinely ambiguous rather than me just not knowing the codebase well enough. I'd want to be ready to argue it, not just state it.

12. Git practices

Q: Why this commit structure? Seven commits, each scoped to one coherent unit of work matching how the brief's own example commit messages are shaped (schema → activity module → assignment logic + the bug fix together since they touched the same file → seed backfill → frontend → tests → docs). Realistically, in day-to-day work I'd commit more incrementally *while* writing each piece rather than in large after-the-fact chunks — worth

saying if asked directly, rather than implying this is how I'd always work. This was reconstructed as a clean, reviewable history after building the feature end to end.

Gotcha questions — quick-hit prep

Short answers for a lightning round; each links back to a section above for the full version.

- **"Is `findOneAndUpdate` really atomic without a transaction?"** — Yes, for a single document, always, on any `mongod` topology. See §6.
- **"What if two people assign the same task at the same moment to different people?"** — Last-write-wins; no uniqueness invariant is being protected (unlike task numbering), so this is fine. Whoever's request reaches Mongo second wins, and the loser's optimistic UI update gets corrected on `onSettled` refetch. Not the same problem as concurrent task creation.
- **"Is the task-update + activity-insert atomic?"** — No, and I'd be upfront that it isn't. See §6, second-to-last answer.
- **"Why not just add multi-document transactions everywhere to be safe?"** — Because no existing code in this repo uses them, they need a replica set, and using them to solve a rare-crash-window problem is more operational complexity than the actual risk justifies right now — the brief's own "avoid overengineering" guidance applies here directly.
- **"What would you do differently with more time on the frontend?"** — Component tests, a "load more" control on the activity timeline, a real accessibility pass beyond the basics I did.
- **"What's the weakest part of this submission?"** — Not having executed the tests myself. Say it plainly if asked — it's already in `AI_LOG.md`, so consistency here matters more than sounding polished.